Testing Report

UrbanHeat Accra — Machine-Learning-Based Urban Heat Risk Prediction & Mitigation-Simulation Dashboard

---

Document Version: 1.0

Date: 15 August 2026

Student Name: Emmanuel Owusu

Student ID: 22425075

Table of Contents

---

1. Testing Strategy

The testing strategy for UrbanHeat Accra focuses on ensuring the reliability and accuracy of the core machine learning inference endpoints, the data retrieval mechanisms, and the input validation layer. Due to the 48-hour development constraint, testing is heavily concentrated on automated API integration tests using `pytest` and FastAPI's `TestClient`.

The strategy follows a black-box approach for the API endpoints, verifying that given specific inputs (e.g., valid environmental features, invalid feature ranges, existing/non-existent location IDs), the system produces the expected HTTP status codes and JSON response structures.

---

2. Testing Scope

2.1 In Scope

- **API Endpoints:** Validation of all REST routes (`/api/health`, `/api/locations`, `/api/predict`, `/api/explain`, `/api/simulate`).

- **Input Validation:** Verification that Pydantic schemas correctly reject out-of-bounds parameters (e.g., NDVI > 1.0).
- **Error Handling:** Verification that the global exception handler prevents stack trace leakage and 404s are handled gracefully.
- **ML Inference Integration:** Verification that the ML model produces valid risk scores (0-100) and that the vegetation simulation directionally reduces risk.

2.2 Out of Scope

- **Automated Frontend UI Testing:** (e.g., Selenium/Cypress) due to time constraints. Frontend was tested manually.
- **Unit Testing of ML Training:** The model training script (`train_model.py`) is a one-off utility and is excluded from the test suite.
- **Performance/Load Testing:** Render and Vercel free tiers impose variable latency; rigorous load testing is deferred to future phases.

---

3. Test Environment

- **Operating System:** Windows/Linux/macOS (Cross-platform)
- **Python Version:** 3.10+
- **Test Framework:** `pytest` $\geq$ 8.3.3
- **HTTP Client:** `fastapi.testclient.TestClient`
- **Database:** SQLite (`urbanheat.db`) — dynamically populated with a test record during test setup.

---

4. Functional Testing

Functional testing was performed to verify that the system meets its primary functional requirements (FR1-FR8). The automated test suite (`backend/tests/test_api.py`) covers the backend functionality. Frontend functionality was verified through manual exploratory testing.

---

5. Integration Testing

Integration testing focuses on the interactions between:

- The FastAPI routes and the SQLAlchemy Database Session (e.g., fetching locations).
- The FastAPI routes and the loaded `scikit-learn` model (e.g., predicting risk from inputs). Both are exercised simultaneously in the `pytest` suite via the `TestClient`.

---

6. System Testing

System testing evaluated the end-to-end application by running the frontend React SPA against the backend API running on `localhost:8000`, verifying CORS policies, data parsing, and UI rendering of API responses.

---

7. User Acceptance Testing

User Acceptance Testing (UAT) criteria were defined against the project requirements. The system is considered acceptable if a non-technical user can successfully view the map, inspect a location's risk factors, and run a vegetation simulation without encountering unhandled errors.

---

8. Test Cases and Results

The following table details the automated test cases executed against the backend API.

Test ID	Requirement ID	Test Case	Preconditions	Steps	Expected Result	Actual Result
TC1	NFR8	Health check	Backend running	GET `/api/health`	HTTP 200, `{"status": "ok"}`	**Not verified — execution required by student**
TC2	FR1, FR5	List locations	Database seeded	GET `/api/locations`	HTTP 200, JSON array of locations	**Not verified — execution required by student**
TC3	FR5, FR8	Predict valid	ML loaded	POST `/api/predict` with valid features	HTTP 200, risk_score 0-100, valid category	**Not verified — execution required by student**
TC4	NFR6	Predict invalid	ML loaded	POST `/api/predict` with NDVI = 5.0	HTTP 422 Unprocessable Entity	**Not verified — execution required by student**
TC5	NFR6	Unknown loc.	DB seeded	GET `/api/locations/999999`	HTTP 404 Not Found	**Not verified — execution required by student**

TC6	FR3, NFR7	Explain factors	DB seeded	GET `/api/explain/{id}`	HTTP 200, top_factors array populated	**Not verified — execution required by student**
TC7	FR4	Simulate delta	DB seeded	POST `/api/simulate` with delta = 20	HTTP 200, after_risk_score <= before_risk_score	**Not verified — execution required by student**
TC8	NFR6	Simulate bad body	Backend running	POST `/api/simulate` `{"location_id": "text"}`	HTTP 422 Unprocessable Entity	**Not verified — execution required by student**

---

9. Defects and Corrective Actions

To be populated by the student upon execution of the test suite.

Defect ID Associated Test Description Severity Status Corrective Action

---

10. Requirements-to-Test Traceability

Requirement	Description	Test Cases
FR1	Display locations on map	TC2 (Backend data source)
FR3	Contributing factors	TC6
FR4	Vegetation simulation	TC7, TC8
FR5	RESTful API	TC1-TC8
FR8	Custom prediction	TC3, TC4
NFR6	Input validation	TC4, TC5, TC8
NFR7	Explainability	TC6
NFR8	Health check	TC1

---

11. Testing Summary

The automated testing framework provides solid coverage of the critical path APIs, input boundary validation, and error state handling. Upon execution and verification by the student, the system's backend stability will be demonstrably proven against the requirements.